Batch 1277 — Timeout Investigation (4 jobs at the 2100 s pipeline limit)

Batch 1277 remains FAILED / INCOMPLETE. Nothing was deployed, re-run, or re-tuned during this investigation.

Branch: `fix/browser-lifetime-and-stop-rule` (base `ba9a2ce`, no merge or deploy). Production evidence was read only:
- prod logs `v25run/logs.jsonl`, 7,387 lines;
- `companyiq_v3` queried SELECT-only with `default_transaction_read_only=on`;
- no Redis writes.

Every local repro used saved corpora and synthetic inputs, with no paid providers.

Job	Company	id	Outcome
96945	DARDEN	14	SIGKILL at 17:30:57 (pipeline_deadline 2100 s)
96950	CELESTICA	19	SIGKILL at 17:30:57
96955	AXA	24	SIGKILL at 17:30:56.9
96960	Idemitsu	29	SIGKILL at 17:30:57.007

All four were claimed at about 16:55:57. All times in this report are UTC on 2026-10-12, taken from the prod log timestamps.

---

1. Ranked causes

#1 Dominant (measured): abandoned document operations keep, or later take, the replica’s single browser permit, and recovery steps inside the permit have no deadline

The caller’s timeout stops the caller from waiting. It does **not** stop the operation. The operation then keeps the per-replica browser permit (`MAX_CONCURRENT_BROWSER=1`, brokered by the parent). If the operation is still queued when its caller gives up, it still acquires the permit later and dispatches. Inside the permit, the PDF-recovery and browser-fetch steps are bounded only between URLs. A single URL step can therefore run for many minutes. Every other company on the same replica that needs the browser is then serialised behind it until it is killed.

Code locations (exact):

File : line	Function	Defect
`server/lib/pipeline.ts:179`	`withTimeout()`	Uses `setTimeout` and then `reject`. It never cancels the inner promise and has no AbortSignal.
`server/lib/pipeline.ts:1135`	first-pass per-doc fetch	Calls `withTimeout(processDocument(url, type, …), PER_DOCUMENT_TIMEOUT_MS)`. The work outlives the timeout.
`server/lib/pipeline.ts:1701`	CHANGE 5 deferred-retry sweep	Calls `withTimeout(processDocument(url, type), min(PER_DOCUMENT_TIMEOUT_MS, remaining))`. The work outlives the 120 s sweep.
`server/lib/processor.ts:846`	`acquireBrowserSlot()`	`remoteAcquire("browser")` takes no signal or deadline. A queued waiter whose caller has given up still receives the permit. `browserDispatchRefused()` (`:874`) checks only attempt-level revocation, not per-operation deadlines.
`server/lib/processor.ts:1660`	`fetchPdfViaBrowser()`	Holds the permit across `page.goto`, `resp.buffer()`, `extractTextFromPdf()` (pdf-parse, then pdftotext with a 60 s timeout, then OCR), `primeWafSession`, and `inPageFetch` (an in-page `fetch` with no timeout, bounded only by puppeteer `protocolTimeout` = 120 s per call), then re-primes and runs a second `inPageFetch`. The permit is released only in `finally`.
`server/lib/processor.ts:1845`	`fetchIssuerPdfsWithPrimedSession()`	One permit covers a whole origin. The budget is `min(600 s, max(180 s, 15 s`

File : line	Function	Defect
		`× URLs))` and is checked only at the top of each URL (`:1964`). The C/A/B strategies and the `extractTextFromPdf` inside a URL have no deadline. The budget clock runs during queue wait for admission.

Measured evidence that this cause dominates (prod logs):

- **Replica B, Idemitsu holds the permit for 18 min 10 s.**
- Child 48 last acquired the permit at 17:12:46.572 and launched Chromium pid 3471 at 17:12:46.715.
- It then logged nothing: no release, no close.
- SIGKILL came at 17:30:57.007. **OTSUKA's child 1492 acquired the permit at 17:30:57.054, 47 ms after the kill.**
- The last state Idemitsu logged was "2 origin(s) queued for browser recovery" (17:11:58), so the hold happened inside Fix 2b recovery or `fetchPdfViaBrowser`.
- Its preceding cycles were short. It fetched a 27.6 MB PDF (`report2018_web_all.pdf`) and released at 17:12:00.373, then held for 12 s and for 26 s.
- Child stdio is inherited (`job-isolation.ts:251` , `stdio: ["inherit","inherit","inherit","ipc"]`), so the silence is not log buffering lost at the kill. The child really produced no output for 18 min.
- **Darden waited behind that hold.** At 17:12:49.632 it logged "HTTP 403 … trying browser fallback". It got no permit and logged nothing further until SIGKILL. Earlier, at 17:12:46.572, its recovery for the `liptonfinancialservices` origin logged "overall budget (180s) exhausted" **with 0 URL attempts**. That budget had been spent queueing for admission.
- **Celestica waited behind that hold.** Its last browser use was at 17:11:06.887 (recovery with Firecrawl ended at 17:11:07). It then waited for the permit until SIGKILL.
- **The same replica's next tenant shows the queued-timeout pattern.**
- OTSUKA logged 15 admission waits (union 960,983 ms; summed waiter time 11,109,763 ms; navigation only 10,383 ms).
- Its "Document fetch TIMEOUT (150s)" lines fired while those waits were still queued. The abandoned queued acquires stayed in the queue.
- **Replica A, abandoned deferred retry holds the permit for 5 min 55 s.**
- BCE's deferred retry logged "timed out after 120s" at 17:13:24.525, and the sweep completed.
- Child 31 then released and **re-acquired** the permit at 17:13:27.500 and held it until 17:19:22.630, while BCE itself was already in PHASE 2.
- Over the same window, admission-wait unions on replica A were CSX 1,002,585 ms, Dominion 887,490 ms and BCE 573,067 ms.
- AXA's fetch phase ran to "BUDGET EXCEEDED" (1651 s) at 17:23:29.
- **Browser launch/close is not the cost.** Close plus reap took about 60–205 ms and launch about 0.1–0.5 s. The browser-lifetime change on this branch is not on the critical path, in either direction.

Local mechanism repro (`evidence/timeout1277_permit_hold.ts` , output in `evidence/timeout1277_permit_hold.out.txt`):

- It uses a verbatim copy of `withTimeout` and the hand-off semaphore semantics, with MAX = 1.
- **Scenario 1, timeout while holding:**
- A's caller times out at 202 ms.
- A keeps the permit until 1502 ms and "persists" its result.
- Company B waits **1492 ms** for admission.
- **Scenario 2, timeout while queued:**
- Q's caller times out at 150 ms while Q is still queued.
- Q is still **granted the permit later (2354 ms), dispatches, and persists**.
- B waits **1392 ms**, including the whole of Q's dead work.

What is still hypothesis: the exact step Idemitsu was blocked in during its 18 min hold. Candidates, all consistent with the code:
- an `inPageFetch` `page.evaluate` (no fetch timeout; 120 s protocolTimeout per call, repeated per strategy and per URL);
- `extractTextFromPdf` on a large PDF (pdf-parse is synchronous CPU, then pdftotext and OCR);
- a `primeWafSession` stall.

The proposed incremental lease records (§3) would settle this on the next run.

#2 Secondary (measured stall; mechanism unconfirmed): AXA's analysis preparation, 7 min 26 s with no output

- AXA logged "Analyzing with 43 fetched documents" at 17:23:30.7. It never logged "ABOUTNESS corpus" before SIGKILL at 17:30:56.9.
- Between those two log lines, `pipeline.ts:2199–2364` runs:
- `updateCompany`;
- `resolveCikForCompany`, which returns null immediately for AXA because its ISIN is FR, so there is no network call;
- `storage.getCompanies(workspaceId)`, which loads 2,678 peers;
- per document: `scoreAboutness()` and `classifyProvenance()`, with an awaited `storage.updateDocumentSourceType()`.
- **Local replay of this loop on AXA's saved batch-1277 corpus** (43 docs, 35.4 M chars, 2,675 peers; `evidence/timeout1277_analyze_prep.ts`) takes **10.1–10.8 s**, not 7.5 min. The `node --cpu-prof` profile (`evidence/timeout1277_profile_summary.txt`) shows `scoreAboutness` at 85.7% inclusive and `countTerm` at 84.1%: one RegExp is built and executed per peer, over a 400 k-char slice per document.
- The loop is O(docs × peers), which is real cost, but the local measurement **does not explain** the prod stall.
- The same section was slow in prod for companies that completed: BCE took 5 min 29 s (local 9.5 s) and Dominion 6 min 38 s (local 13.7 s). By contrast, CSX took 43 s (local 9.3 s) and MAGNA 37 s.
- Hypotheses, not yet distinguished:
- (a) CPU starvation of children on the replica. Up to 4 children plus Chromium were running; Dominion's aboutness loop overlapped AXA's window.
- (b) Slow per-document DB awaits, or the `getCompanies` await.
- (c) GC or memory pressure from 35 M chars of corpus in one child.
- Healthy parent event-loop lag (≤ 14 ms) says nothing about the children.

#3 Secondary (measured, bounded): serial discovery

Discovery took Darden 464 s, Celestica 428 s, AXA 494 s and Idemitsu 579 s (about 210–234 searches each, run serially), finishing by about 17:05:40. This is fixed overhead that reduces headroom. It did not cause the kills on its own: each job still had more than 25 min left when discovery ended.

Not established

- **LLM-slot contention:** not supported. The killed jobs never reached scoring. `LLM_MAX_CONCURRENCY` should not be raised.
- **The BM25 TF recomputation** (`passage-retrieval.ts bm25Score()`) is a valid defect with prior profile support (`docs/worker-isolation/PROFILE_EVIDENCE.md`). It is **not** on the critical path for these four jobs, because none of them reached passage retrieval.

---

2. Per-job time attribution (2100 s window, approx. 16:55:57 → 17:30:57)

Job	Discovery	Fetch + first-pass recovery	Browser-permit segment	Analysis prep	Dominant loss
DARDEN 96945	~464 s	~550 s (to 17:12:46; includes queue-consumed 180 s recovery budget)	**~1,088 s waiting** for the permit Idemitsu held (17:12:49 → kill)	0	#1 (waiting behind the abandoned hold)
CELESTICA 96950	~428 s	~480 s (to 17:11:07)	**~1,190 s waiting** (17:11:07 → kill; from 17:12:46 behind Idemitsu's hold)	0	#1 (waiting)
Idemitsu 96960	~579 s	~430 s (to 17:12:46)	**~1,090 s holding** the permit in an unbounded recovery/ browser step	0	#1 (holding)
AXA 96955	~494 s	~1,159 s (17:04:11 → 17:23:30; the "BUDGET EXCEEDED" figure of 1,651 s is counted from pipeline start); includes admission waits behind BCE's abandoned 5 min 55 s hold	included in fetch	**~446 s with no output**	#1 (fetch) + #2 (prep stall)

The segments are taken from log timestamps. Pipeline-level stage timers are emitted only on completion, so the killed jobs have no per-stage JSON. For the same reason, no batch-corpus snapshot exists for Darden, Celestica or Idemitsu (0 rows in batch 1277). Only AXA froze a corpus.

---

3. Smallest corrective proposal (described only, NOT implemented or deployed)

Operation-scoped cancellation for document operations, wired into the browser permit. Existing budgets stay unchanged.

1. Replace `withTimeout(processDocument(...))` at `pipeline.ts:1135` and `:1701` with an operation `AbortController`. Combine its signal with the attempt-lifecycle signal (`AbortSignal.any`). On timeout, abort; do not merely reject.
2. Pass the signal to `processDocument` → `fetchPdfViaBrowser` / `fetchIssuerPdfsWithPrimedSession` → `acquireBrowserSlot(signal)` .
   - In `acquireBrowserSlot` , a waiter whose signal is aborted (or whose deadline has passed) is removed from the queue, both in-process and via a `remoteCancel` to the parent broker. A grant that arrives after an abort is released immediately and **never dispatches**.
3. While holding the permit, bound each step by the remaining operation deadline:
   - `page.goto` ;
   - `page.evaluate` for `inPageFetch` (with an in-page `AbortController` timeout);
   - `primeWafSession` ;
   - `extractTextFromPdf` (pass the deadline to pdftotext/OCR).

On abort, close the page and release in `finally` . A late result is never persisted, which means the persist step checks `signal.aborted` .
4. In `fetchIssuerPdfsWithPrimedSession` :
- start the per-origin budget **after** admission;
- check the remaining company deadline before and after admission;
- never start once the deadline has expired while queued;
- keep URLs not attempted explicitly *pending/incomplete*. No URL is dropped and coverage rules stay binding.
5. Do not start a second recovery or retry of the same URL while an earlier execution of it is still active.
6. Alongside: write incremental lease records (owner job/attempt/op, origin, URL count, `queued_at` , `granted_at` , `released_at` , outcome) on acquire, release and settle. Write them synchronously to stdout or through the IPC channel, so they survive a child SIGKILL.

---

4. One bounded validation test

A deterministic local test (vitest, local HTTP server, synthetic responses, `MAX_CONCURRENT_BROWSER=1` , completes in < 10 s):

1. **Timeout while holding:** company A’s op acquires the browser, and the synthetic server delays the PDF response for 60 s. The op deadline is 500 ms.
   - The op must abort and the permit must be released within **≤ 1 s** of the deadline.
   - The page must be closed.
   - No `recordFetchSuccess` or persist may happen, even after the server responds later.

2. **Timeout while queued:** op Q is queued behind A, with a 200 ms deadline.
   - Q must leave the queue at its deadline.
   - When A releases, Q must **never** be granted the permit and must never dispatch.
3. **Next tenant admitted:** company B's op is queued after Q. It must acquire the browser within **≤ 1 s** of A's abort, not after A's natural completion.
4. **Recovery under a queue-expired deadline:** `fetchIssuerPdfsWithPrimedSession` entered with a company deadline that expires while it is queued.
   - It returns with 0 attempts and every URL marked pending/incomplete, not dead.
   - The 180 s budget is not reported as exhausted.

The current code fails (1)–(3). The synthetic run above shows a 1,492 ms and a 1,392 ms admission delay, and a dispatch after the caller had already given up.

---

5. Alignment with the external review of `8c03b07`

Review point	Our finding
§1 `withTimeout` does not cancel the operation	**Confirmed by measurement**: BCE re-acquired and held the permit for 5 min 55 s after the 120 s timeout; Idemitsu held it for 18 min 10 s, with a 47 ms hand-off at the kill; OTSUKA's 150 s timeouts fired while queued. This is ranked #1.
§2 recovery is long serial occupancy	**Confirmed and refined**: the recovery budget is consumed by queue wait (Darden: 180 s exhausted with 0 attempts), and steps within one URL have no deadline. This is folded into #1 and proposal items 3–4.
§3 BM25 recomputation	Valid defect, but not on these four jobs' critical path. New finding: `scoreAboutness` is O(docs × 2,675 peers) RegExp (about 10 s per company locally, 84% in `countTerm`).
§4 pre-scoring LLM waits	Not attributed. The effective translation, terminology and semantic flags were **not** verified in this step. AXA's prep stall occurs *before* these stages (it never logged "ABOUTNESS corpus").
§5 LLM-slot hypothesis	Agreed: not established. No concurrency change.
§6 order	Agreed: operation lifetime first, then the BM25 cache, with incremental lease records alongside.

6. Separate notes (not fixed here)

- **Counter semantics:** `failed_jobs = 0` counts terminal failures before cancellation, while the stop rule counts jobs with any failed attempt. The two disagree for this batch. Batch counters should expose *completed / timed_out / failed / cancelled / unstarted* separately.
- **pid-3471 reap: UNVERIFIED.**
- pid 3471 is the Chromium launched by Idemitsu's child 48 at 17:12:46.715. That child was holding the replica-B browser permit when it was SIGKILLed at 17:30:57.007.
- No host process evidence (ps, cgroup, or reaper log) confirms the pid was reaped. Without such evidence it remains unverified.

7. Supporting evidence

7.1 Timelines (prod logs)

Replica B

Time	Event
17:07:42	Celestica "All docs resolved: 45 ok, 3 dead"
17:08:18	Darden 38 ok / 4 dead
17:11:06.887	Celestica last browser use; then waits
17:11:58	Idemitsu "2 origin(s) queued for browser re-covery"
17:12:00.242 / .373	Idemitsu 27.6 MB PDF fetched, release
17:12:00.544 → 17:12:12.619	Idemitsu hold
17:12:12.619 → 17:12:38.813	Idemitsu hold
17:12:46.572	Idemitsu acquire (never released). Darden Fix E budget_exhausted; Darden recovery for lipt-onfinancialservices: 180 s exhausted, 0 at-tempts
17:12:46.715	Idemitsu launches pid 3471
17:12:49.632	Darden "HTTP 403 … trying browser fallback" → waits
17:30:57.007	SIGKILL (Darden, Celestica, Idemitsu)
17:30:57.054	OTSUKA child 1492 acquires permit

Replica A

Time	Event
17:03:21.6 → 17:03:58.6	MAGNA aboutness section 37 s
17:13:24.525	BCE deferred retry “timed out after 120s”
17:13:27.500 → 17:19:22.630	BCE child 31 holds permit (5 min 55 s); no browser launch until 17:17:38.975
17:13:27.9 → 17:18:56.9	BCE aboutness section 5 min 29 s
17:20:33.5 → 17:27:11.3	Dominion aboutness section 6 min 38 s
17:23:29	AXA fetch BUDGET EXCEEDED (1651 s)
17:23:30.7	AXA “Analyzing with 43 fetched documents”
17:30:56.9	AXA SIGKILL with no ABOUTNESS line

Adaptive concurrency dropped to 2 at 17:02:24 and returned to 4 at 17:21:39.

7.2 Profile summary

The full summary is in `evidence/timeout1277_profile_summary.txt`. It was produced with:

```
node --cpu-prof --import tsx timeout1277_analyze_prep.ts 24
```

Metric	Value
Harness wall	10,804 ms
`scoreAboutness` inclusive	10,710 ms (85.7%)
`countTerm` inclusive	10,516 ms (84.1%)
corpus read	458 ms
GC	100 ms

7.3 Repro measurements

Analysis-prep replay on batch-1277 corpora, 2,675 peers, local 2 vCPU:

Company	Docs	Chars	Local wall	Prod section
AXA (24)	43	35.4 M	10,089–10,804 ms	≥ 446 s, never finished
BCE (15)	44	4.12 M	9,512 ms	329 s
Dominion (20)	32	11.88 M	13,719 ms	398 s
CSX (9)	30	3.02 M	9,277 ms	43 s
AXA, 100 peers	43	35.4 M	537 ms	—

Permit-hold mechanism: see §1 and `evidence/timeout1277_permit_hold.out.txt`.

7.4 Corpus availability

The saved corpora were dumped read-only to a VM-local scratch directory outside the repo, and were not committed. Darden, Celestica and Idemitsu have **no** batch-1277 corpus snapshot, because they were killed before freezing one. For them, a before/after comparison on their own corpora requires a fresh non-prod fetch.